

Нечеткая логика для обработки лингвистических неопределенностей

Семинар
"Обработка естественных языков"
осень 2025

Щеглов Михаил 3381

Уфимцева Алиса 3381

Формулировка задачи

Задача: интерпретация отзыва в интернете

Проблема оценки отзывов:

- Тональность в отзывах редко «чёрно-белая»: есть полутона и неопределённость.
- Фразы типа «скорее норм», «вроде неплохо», «не ужасно, но...» ломают бинарные решения (например, классификацию).
- Хотим не только итоговый score, но и интерпретируемые качества: доверие/уверенность, сила впечатления, интенсивность эмоций.

Нечеткая логика

Нечеткая логика - это мост от языка человека к формальному решению.

- В реальных задачах люди пишут не числа, а качественные оценки: «вроде нормально», «скорее плохо», «очень понравилось, но...»

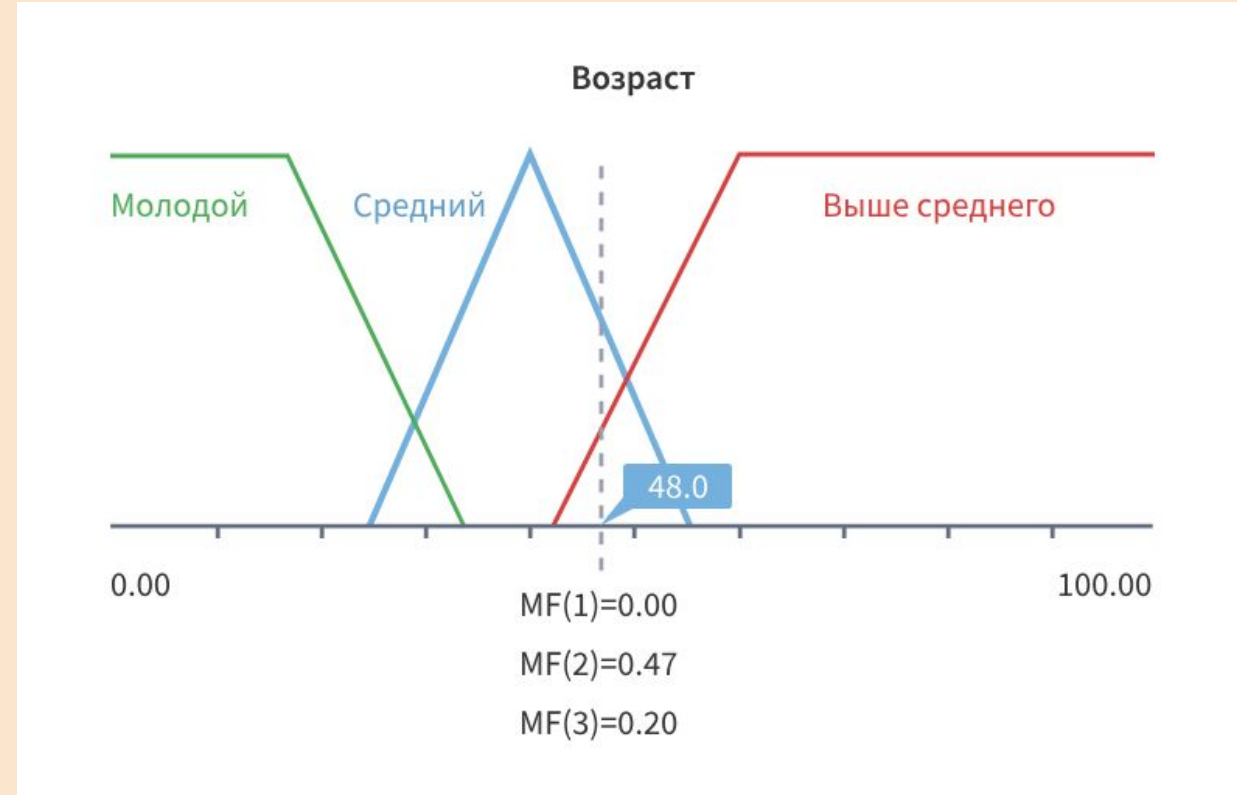
Классическая логика и “жесткие пороги” плохо ловят такие оттенки.

- Нечёткая логика позволяет перевести слова в математику: каждому терму (низко/средне/высоко) ставится в соответствие степень принадлежности (число от 0 до 1).

Поэтому “среднее” и “высокое” могут быть истинны одновременно, но в *разной степени*.

- Дальше решение становится формальным и повторяемым:

мы задаём правила IF / THEN, которые выглядят по-человечески: если **средний возраст высокий** и **возраст выше среднего низкий** -> человек скорее средних лет, чем старый



Основы теории нечетких множеств

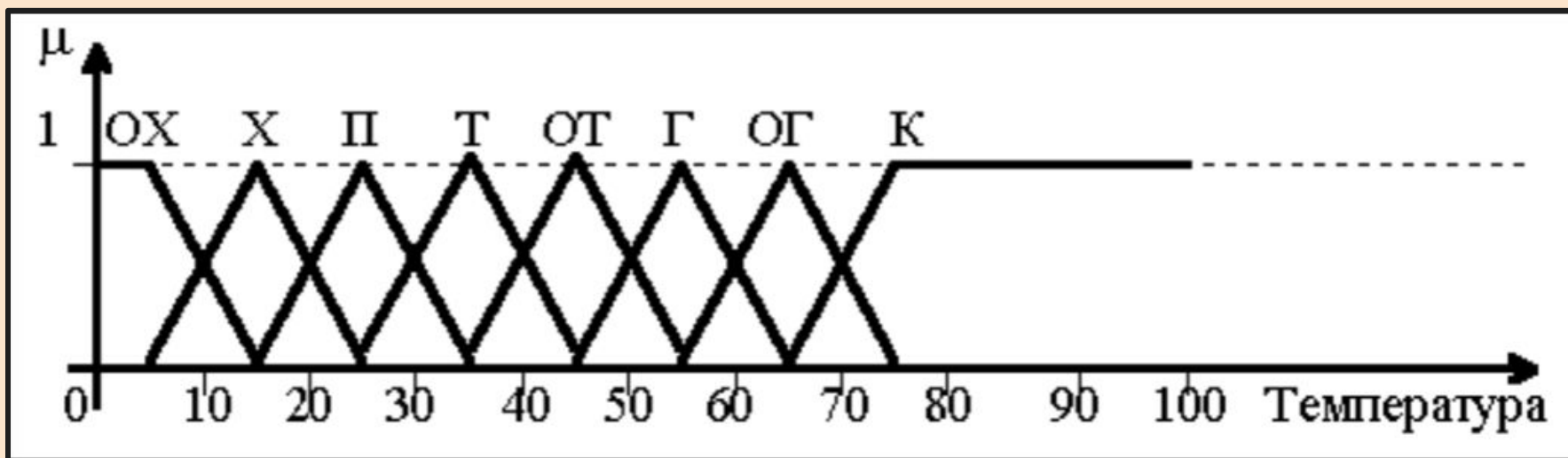
Определение 1: *Лингвистические переменные* - это переменная, значениями которой являются слова или предложения на естественном языке, название самого параметра, для каждого значения которого можно посчитать принадлежность термам нечетких множеств

Определение 2: *Лингвистический терм* - слово или фраза из естественного языка (например, «высокий», «средний»), конкретное описание параметра, к которому считается принадлежность лингвистической переменной. Каждый терм является нечетким множеством.

Определение 3: *Функция принадлежности* для лингвистической переменной - это математическая функция, которая связывает каждое значение из лингвистической переменной со степенью его принадлежности к каждому терму, выражая, насколько точно данный элемент соответствует этому качеству (значение от 0 до 1).

Основы теории нечетких множеств

Пример. Пусть температура жидкости для ванны в диапазоне от 0 до 100 °С описывается лингвистической переменной «Температура» с набором лингвистических термов (значений) «очень_холодная» (ОХ), «холодная» (Х), «прохладная» (П), «теплая» (Т), «очень_теплая» (ОТ), «горячая» (Г), «очень_горячая» (ОГ), «кипяток» (К).



Подзадачи

1. Генерация словаря нейросетью.
2. NLP-обработка конкретного отзыва.
3. Нечёткая логика для интерпретации отзыва.



Генерация словаря

Основная цель: построить большой словарь лемм с оценочной полярностью $\text{polarity}(\text{lemma}) \in [-1; 1]$.

Зачем нужен словарь: перевести “чёрный ящик” оценки тональности в **интерпретируемые признаки** на уровне слов (лемм): у каждой леммы есть полярность и частота.

Пайплайн: отзывы → предложения → лемматизация → оценка предложения моделью → накопление статистики по леммам → фильтрация → сохранение.

Вход: один или несколько .txt файлов или директории с .txt.

Выход:

- lexicon.json: lemma -> {polarity, freq}
- lexicon.csv: столбцы lemma, polarity, freq (с сортировкой)

Генерация словаря (модель)

Модель сентимента и её параметры:

- Это русскоязычная модель для коротких текстов, специально заточенная под задачу сентимента.
Она решает задачу в формате 3 классов: negative / neutral / positive.
- rubert-tiny — компактная базовая модель, её обычно выбирают, когда важна скорость без ухода в “тяжёлую артиллерию”.

Как получаем оценку предложения:

- Модель читает предложение и выдаёт три уверенности: $P(\text{негатив})$, $P(\text{нейтр.})$, $P(\text{позитив})$.
- Мы превращаем их в одно число по простой шкале: негатив = -1 , нейтрально = 0 , позитив = $+1$.
- Итоговый балл предложения = “средний ответ модели” по этой шкале: $\text{score} = (+1) \cdot P(\text{позитив}) + 0 \cdot P(\text{нейтр.}) + (-1) \cdot P(\text{негатив})$

Перенос оценки на одну лемму:

в каждом предложении берём множество уникальных лемм (без повторов), затем для каждой леммы копим: $\text{freq} += 1$ (сколько предложений содержало лемму) и $\text{score_sum} += \text{score}(\text{sentence})$.

Итоговая полярность леммы: $\text{polarity} = \text{score_sum} / \text{freq}$ (это средняя оценка контекстов, где оно встречалось!),
далее фильтры по min_freq и почти нулевым словам.

Проверка словаря на адекватность

Spearman ρ , зачем он нам?

- Что измеряет: Spearman ρ — это показатель того, насколько одинаковый порядок (ранжирование) у двух списков.

Не сравнивает точные числа, а сравнивает: кто “выше”, кто “ниже”.

Короче: это метрика “совпадения рейтинга”.

$$r_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

- Диапазон значений:

- $\rho = +1$ идеальное совпадение рангов
- $\rho = 0$ связи почти нет (порядок случайный)
- $\rho = -1$ обратный порядок (полный перевёртыш)

$$r_s = \rho [R[X], R[Y]] = \frac{\text{cov} [R[X], R[Y]]}{\sigma_{R[X]} \sigma_{R[Y]}}$$

- Почему подходит для словаря: нам важно, чтобы при пересборке словаря самые позитивные и самые негативные слова оставались теми же. Даже если полярности чуть “плавают”, порядок должен сохраняться.
- Если ρ высокий - словарь адекватный и данные/фильтры дают устойчивый результат.

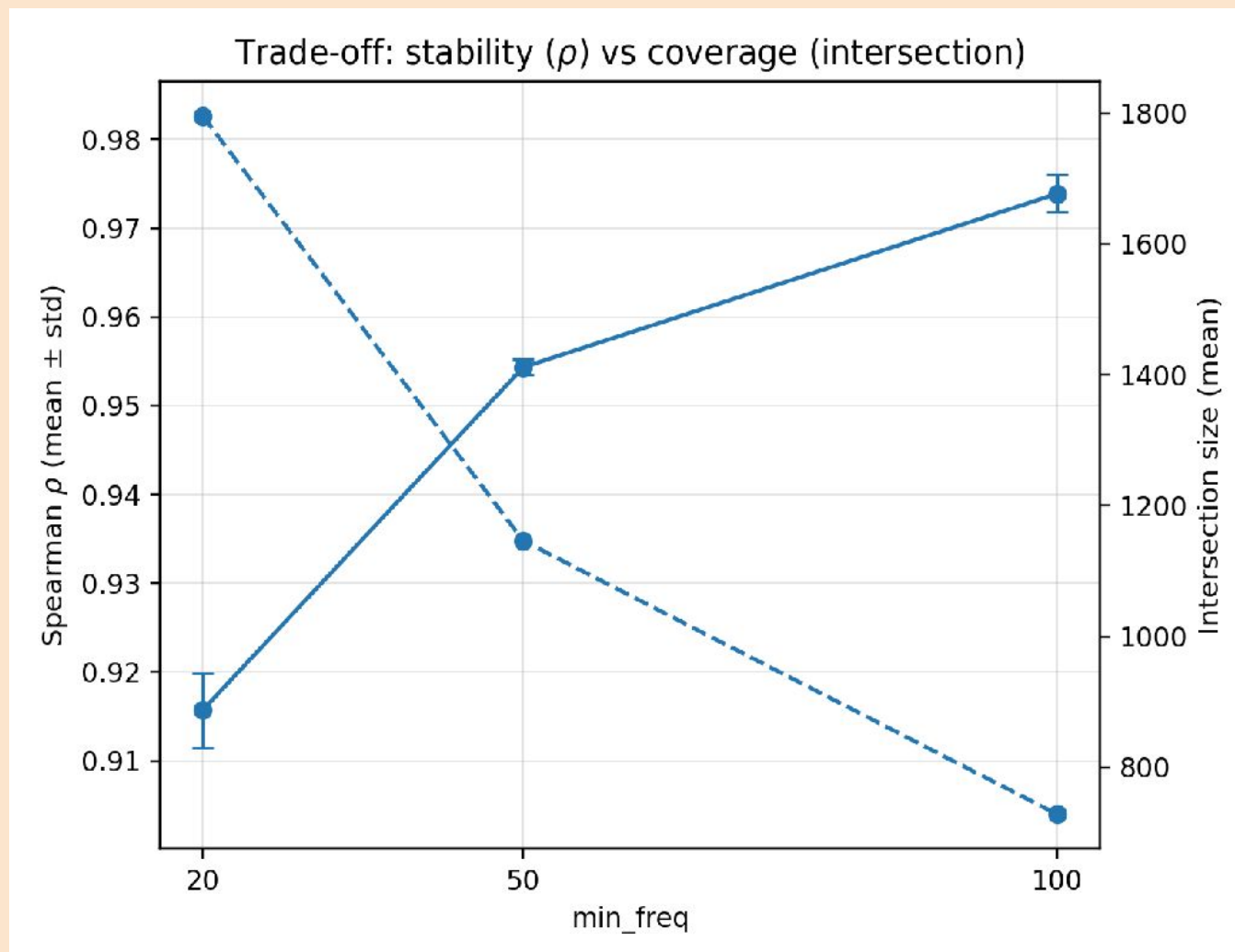
Проверка словаря на адекватность

Этот график показывает торговлю между стабильностью (Spearman ρ) и покрытием (intersection size) при изменении порога `min_freq` в словаре.

Stability (ρ). С увеличением `min_freq` ρ увеличивается от 0,91 до 0,98. Это говорит о том, что с увеличением частоты леммы становятся более стабильными в разных повторениях генерации.

Intersection size (Coverage). Покрытие лемм уменьшается с ростом `min_freq`. Для `min_freq=20` покрытие около 1800 лемм, а для `min_freq=100` уже 900.

Итог: При низком `min_freq` словарь становится более обширным, но менее стабильным. При высоком же словарь меньше, но леммы в нём более стабильны.



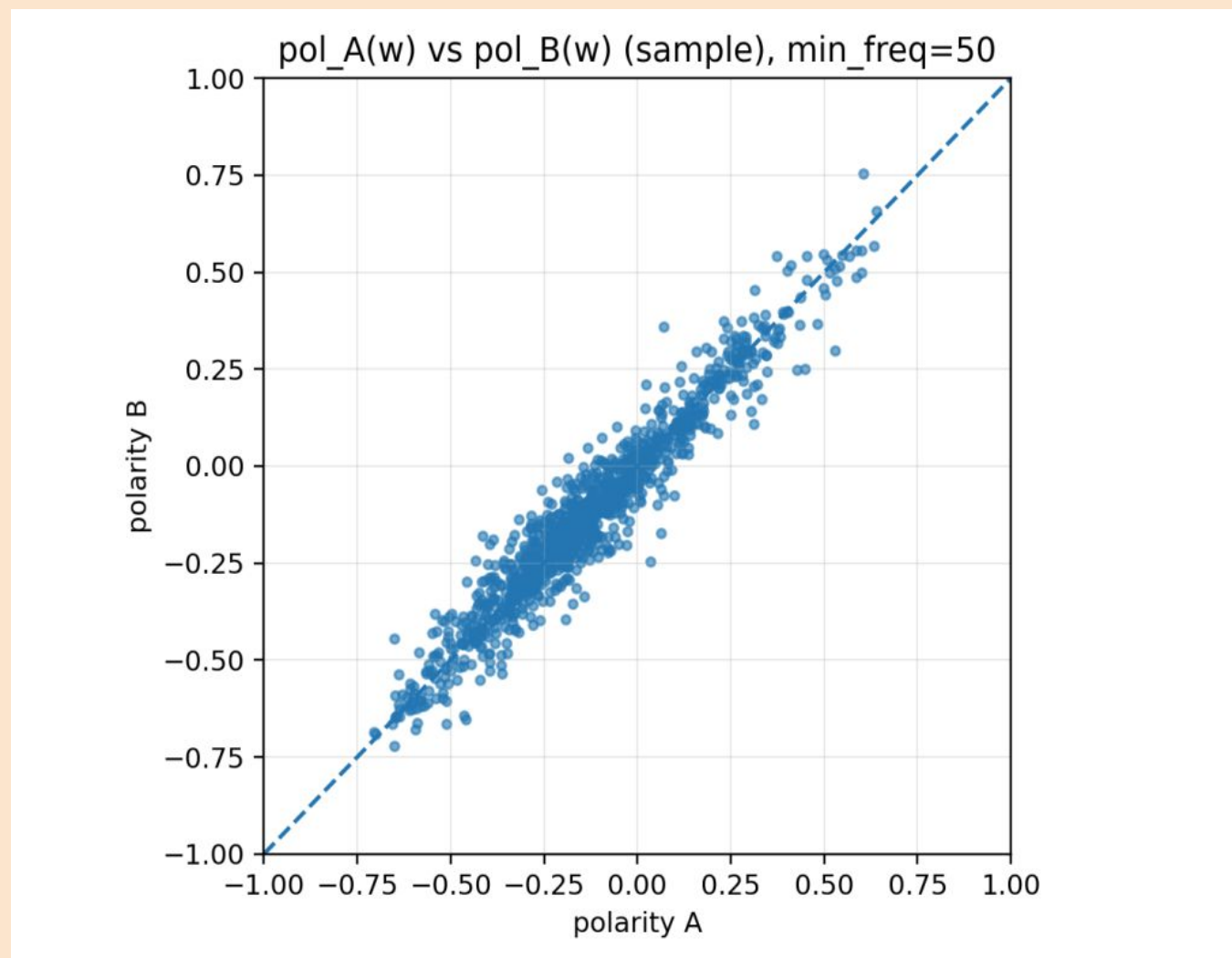
Проверка словаря на адекватность

Каждая точка — **одна лемма**.

- По X: её полярность в прогоне **A**.
- По Y: полярность этой же леммы в прогоне **B**.
- Пунктирная диагональ $y=x$ - идеальный случай:
в A и B слово оценилось одинаково.

Заметим, точки лежат плотной полосой вдоль диагонали, это значит, если лемма позитивная/негативная в одном прогоне, она почти такая же в другом. Словарь воспроизводим, оценка не рандомная.

Итог: две независимые пересборки дают почти одинаковые полярности для одних и тех же слов.

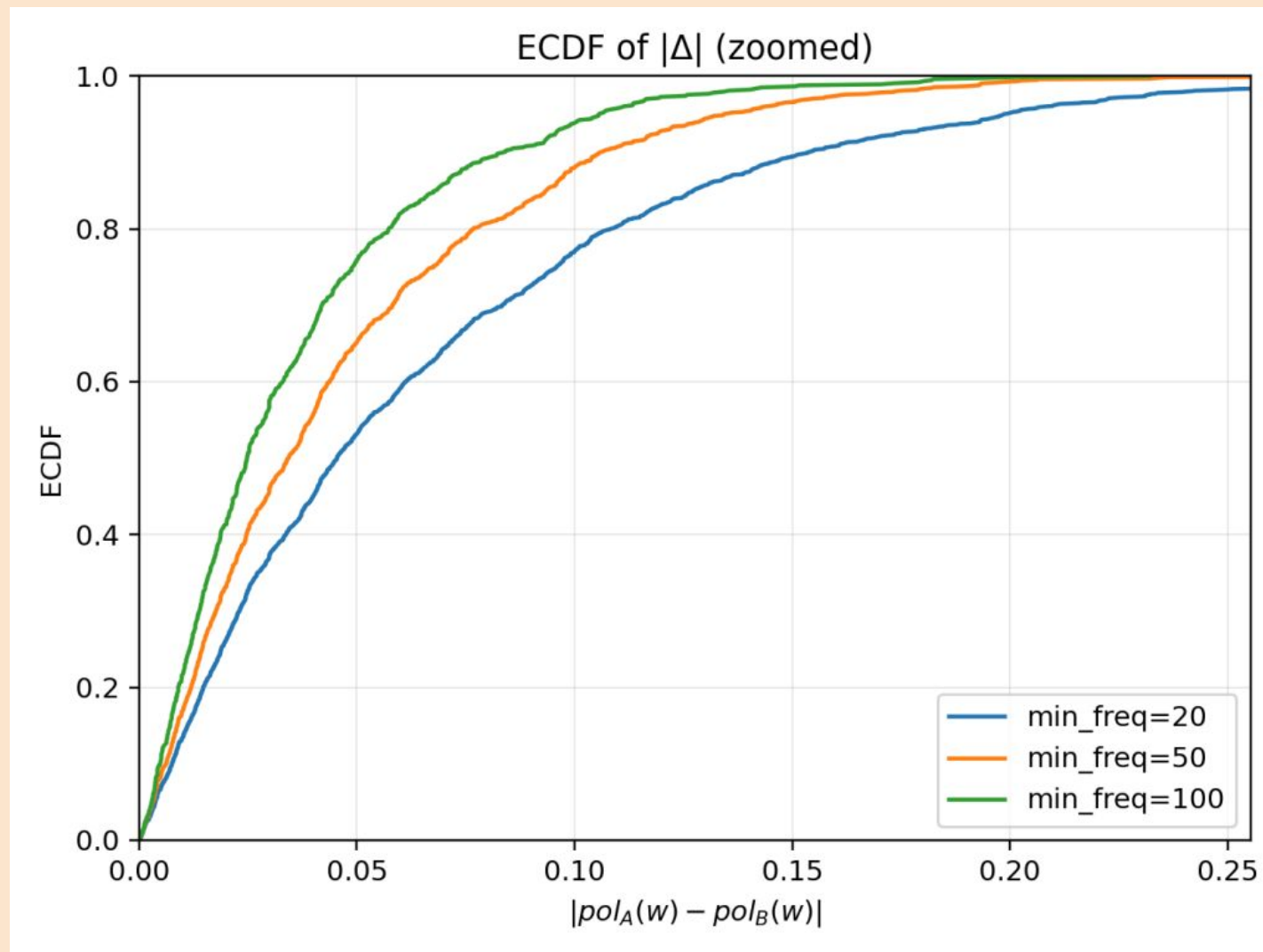


Проверка словаря на адекватность

- Ось X: $|\text{polA}(w) - \text{polB}(w)|$ - разница полярности леммы в двух прогонах
- Ось Y: **доля лемм**, у которых разница $\leq X$ (ECDF)

Берём любой порог по X (например 0,05) и смотрим на Y - это “какой процент слов стабилен больше, чем на 0,05 (чем на значение X)”.

Кривые быстро растут уже на малых X (до ~0,05-0,10). Это значит: у большинства слов полярность меняется совсем незначительно.



Генерация словаря (итог)

1. Построен интерпретируемый словарь: лемма $\rightarrow$ {polarity $\in$ [-1;1], freq}.
2. Полярность леммы получена из контекстов: оценка предложений моделью и усреднение по лемме.
3. Проверено качество: высокая воспроизводимость при пересборке словаря.
4. Итоговый выбор: min_freq = 50 как баланс качества и объёма словаря.

Top negative lemmas:

обманщик	polarity=-0.8733	freq=288
return	polarity=-0.8067	freq=53
кормить	polarity=-0.8041	freq=224
завтрак	polarity=-0.8027	freq=131
истечь	polarity=-0.7935	freq=174
закрыть	polarity=-0.7912	freq=1184
возвращать	polarity=-0.7830	freq=984
недобросовестный	polarity=-0.7805	freq=50
мошенник	polarity=-0.7787	freq=178
обмануть	polarity=-0.7722	freq=232
прождать	polarity=-0.7664	freq=215
обман	polarity=-0.7619	freq=261
обещание	polarity=-0.7572	freq=139

Итого: построенный словарь корректный, потому что при пересборке словаря полярности лемм почти не меняются: ранжирование стабильно (высокий ρ), а большинство слов имеет малую $|\Delta|$, то есть результат воспроизводим, а не случайный. Данных достаточно, потому что при разумном пороге частоты (min_freq=50) стабилизация уже достигает “плато”: дальнейшее ужесточение порога даёт небольшой прирост устойчивости, значит контекстов для оценки лемм хватает.

NLP-обработка конкретного отзыва

Цель: превратить каждый отзыв из текста в набор числовых признаков, которые можно:

1. сравнивать между отзывами,
2. подавать на вход нечеткой системе,
3. объяснять человеку (каждый признак имеет смысл).

Что используем:

- словарь полярностей из этапа 1 (лемма → polarity).
- список модификаторов языка (смягчители, усилители/ослабители, контраст).
- базовую обработку текста (лемматизация).

Вход/выход:

- **Вход:** файл(ы) с отзывами + lexicon.json (словарь) + конфиг модификаторов.
- **Выход:** features.jsonl, где 1 строка = 1 отзыв и набор чисел.

```
{"id": 0, "text": "качество плохое пошив ужасный (горловина наперекос) Фото не соответствует Ткань ужасная рисунок блеклый маленький рукав не такой УЖАС!!!! не стоит за такие деньги г.....", "pos": 1.84943, "neg": 2.6814, "score_crisp": -0.183624, "intensity": 0.266519, "emotion_intensity_crisp": 0.251713, "emotion_intensity_score": 0.251713, "tokens_alpha": 24, "lex_hits": 17, "coverage": 0.708333, "intensifiers_count": 0, "downtoners_count": 0, "hedges_count": 0, "negations_count": 3, "contrast_count": 0}
```

NLP-обработка (числовые характеристики)

1) Тональность по лексикону (ядро)

- pos — суммарная “позитивность” по словам из словаря
- neg — суммарная “негативность” по словам из словаря
- **score_crisp** — итоговая “жёсткая” оценка тональности отзыва (на шкале от отрицательной к положительной)

$$\text{score_crisp} = \frac{\text{pos} - \text{neg}}{\text{pos} + \text{neg} + \varepsilon}$$

2) Сила эмоций

- emotion_intensity_crisp — насколько “эмоционально сильно” написан отзыв (класс/терм)
- emotion_intensity_score — численная оценка интенсивности (число)

$$\text{intensity} = \text{clip} \left(\alpha \cdot \frac{\text{intensifiers_count}}{\max(N, 1)} + \beta \cdot \frac{\text{exclam_count}}{\max(N, 1)} + \gamma \cdot \text{caps_ratio}, 0, 1 \right)$$

NLP-обработка (числовые характеристики)

$$\text{coverage} = \frac{\text{lex_hits}}{\max(N, 1)}$$

$$\text{hedges_rate} = \frac{\text{hedges_count}}{\max(N, 1)}$$

$$\text{magnitude} = 1 - \exp(-(\text{pos} + \text{neg}))$$

$$\text{conflict} = 1 - \frac{|\text{pos} - \text{neg}|}{\text{pos} + \text{neg} + \varepsilon}$$

3) Надёжность/информативность

- **coverage** — доля слов/лемм, которые нашлись в словаре (насколько словарь покрывает отзыв)
- **tokens_alpha** — объём содержательного текста

4) Лингвистическая неопределённость и модификаторы

- **hedges_count**, **hedges_rate** - смягчители/неуверенность («вроде», «как будто», «скорее»)
- учёт negations и contrast («не», «но», «однако»)

5) Производные для этапа 3

- **magnitude** - “сила” оценки (насколько далеко от нейтралы)
- **conflict** - противоречивость (когда в отзыве много и плюсов, и минусов)

Ещё немного теории

Метод Мамдани - это один из первых и наиболее интуитивно понятных подходов к нечеткому выводу, предложенный Ибрагимом Мамдани в 1975 году для управления сложными системами. Он работает, переводя нечеткие входы в нечеткие выходы через правила типа «ЕСЛИ-ТО», объединяя их в одно нечеткое множество, а затем используя дефаззификацию (например, метод центра тяжести), чтобы получить четкий, числовой результат.

Основные шаги метода Мамдани:

1. **Формирование базы правил системы нечеткого вывода.**
2. **Фаззификация входных переменных.** Каждому значению отдельной входной переменной ставится в соответствие значение функции принадлежности соответствующего ей терма входной лингвистической переменной $\mu_1(x)$, $\mu_2(x)$, ..., $\mu_n(x)$, где $\mu_1(x)$, $\mu_2(x)$, ..., $\mu_n(x)$ – функции принадлежности для переменной x .
3. **Агрегирование подусловий в нечетких правилах продукций.** Нахождение степени истинности условий каждого из правил нечетких продукций, используя парные нечеткие логические операции.

Основы теории нечетких множеств

Определение: Фаззификация – процедура нахождения значений функций принадлежности нечетких множеств (термов) на основе обычных (четких) исходных данных. Преобразование исходных числовых физических величин в распределения, соответствующие термам лингвистической переменной. При этом каждое числовое значение описывается одним или несколькими термами, причем его степень соответствия терму задается как степень принадлежности нечеткого множества.

Определение: Деффазификация – это процесс перехода от функции принадлежности выходной лингвистической переменной к её четкому (числовому) значению. Цель дефаззификации заключается в том, чтобы, используя результаты всех выходных лингвистических переменных, получить количественное значение каждой из выходных переменных.

Ещё немного теории

Основные шаги метода Мамдани:

4. **Активизация подзаключений в нечетких правилах продукций.** На этом шаге вес правила применяется к его заключению (консеквенту). В методе Мамдани консеквент, это тоже нечёткое множество.
5. **Аккумуляция заключений нечетких правил продукций.** Все активированные выходные нечёткие множества от всех правил объединяются в одно общее нечёткое множество.
6. **Дефаззификация выходных переменных.** Традиционно используется метод центра тяжести или метод центра площади.



Нечёткая логика (база правил)

Достоверность — 9 правил

Логика: Определяет надежность оценки на основе **охвата (coverage)** и **частоты хеджирования (hedges_rate)**. Высокий coverage повышает trust, большое число хеджей понижает.

Пример: Высокий охват + мало хеджирования → высокая достоверность.

Итоговая тональность — 11 правил

Логика: Корректирует исходную тональность (**score_crisp**) с учетом **coverage**, **hedges_rate**, а также комбинации **magnitude** + **conflict** (смешанные сигналы тянут к нейтрالي).

Пример: Сильная позитивная/негативная + низкий охват → смягчается; много хеджирования → тональность сдвигается к нейтрالي; высокий конфликт при высокой силе → чаще нейтрализация.

Итоговая интенсивность эмоций — 5 правил

Логика: Берет базовую интенсивность из **emotion_intensity_crisp**, затем уточняет с учетом **magnitude** (если “масса” оценки мала, высокая эмоция часто понижается).

Пример: Высокая интенсивность + высокая magnitude → высокая итоговая интенсивность

Сила впечатления — 9 правил

Логика: Определяет, насколько “сильно звучит” отзыв, на основе **magnitude** и **conflict** (противоречивость может усиливать впечатление).

Пример: Высокая magnitude + высокий conflict → высокая сила впечатления

Нечёткая логика (fuzzy-ядро системы)

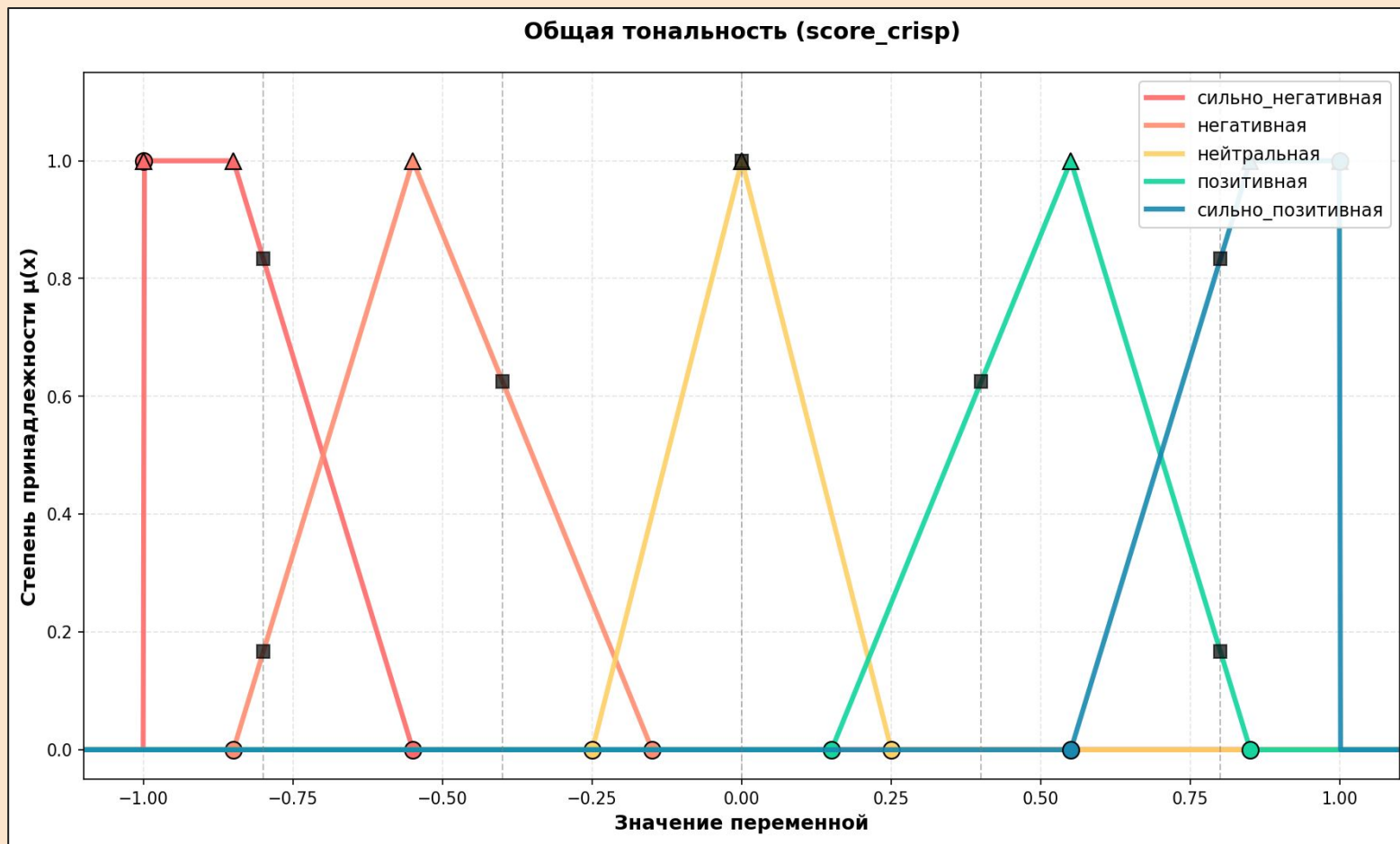
Была определена нечеткая логика для: общей тональности, покрытия лексиконом, частоты хэджеров, интенсивности тональности, уровень конфликтности и интенсивность эмоций

1. score_crisp (Общая тональность)

Диапазон: [-1.0, 1.0]

Термы:

- сильно_негативная: [-1.0, -1.0, -0.85, -0.55]
- негативная: [-0.85, -0.55, -0.15]
- нейтральная: [-0.25, 0.0, 0.25]
- позитивная: [0.15, 0.55, 0.85]
- сильно_позитивная: [0.55, 0.85, 1.0, 1.0]



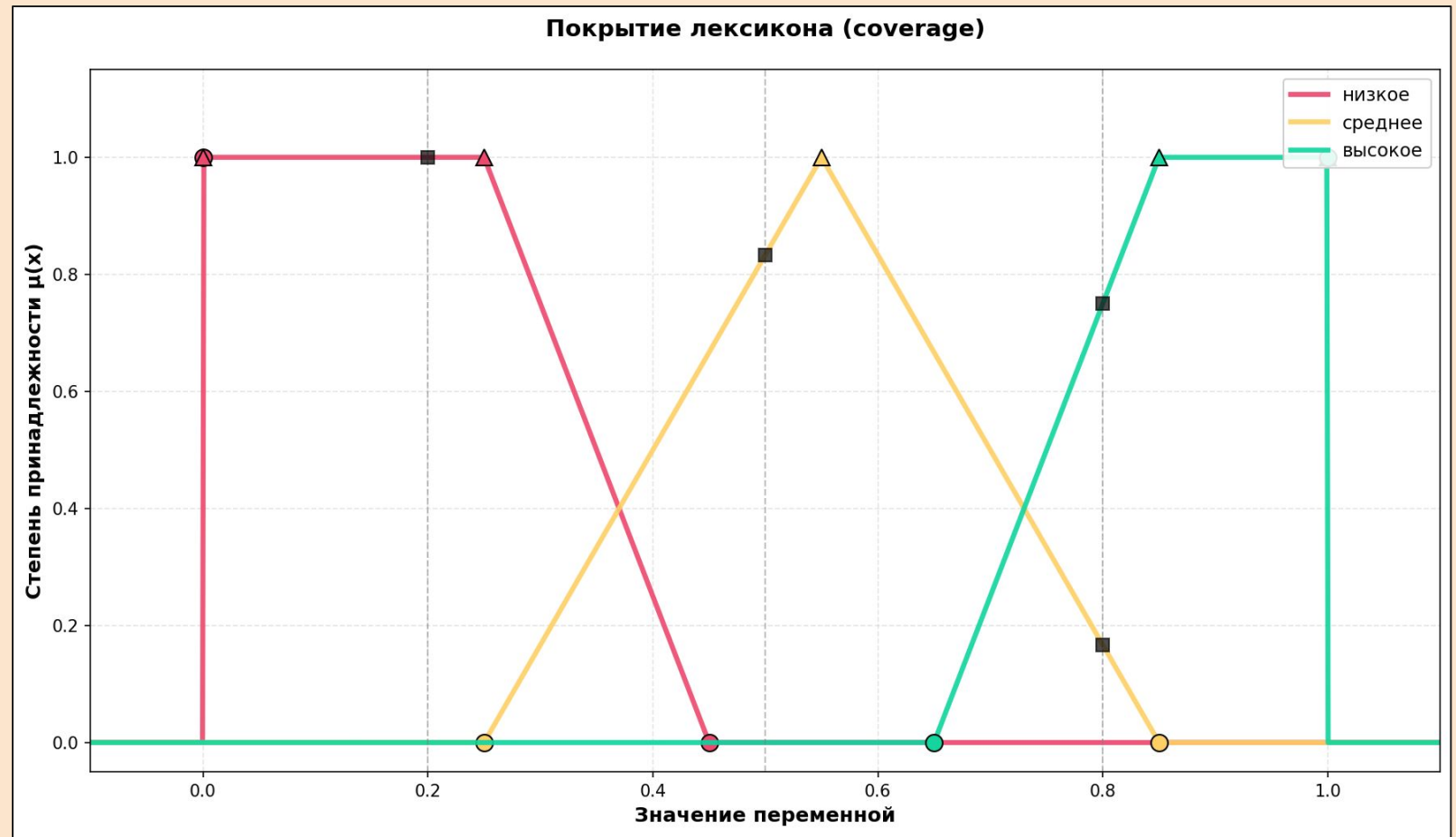
Нечёткая логика (fuzzy-ядро системы)

2. coverage (Покрытие лексиконом)

Диапазон: [0.0, 1.0]

Термы:

- низкое: [0.0, 0.0, 0.25, 0.45]
- среднее: [0.25, 0.55, 0.85]
- высокое: [0.65, 0.85, 1.0, 1.0]



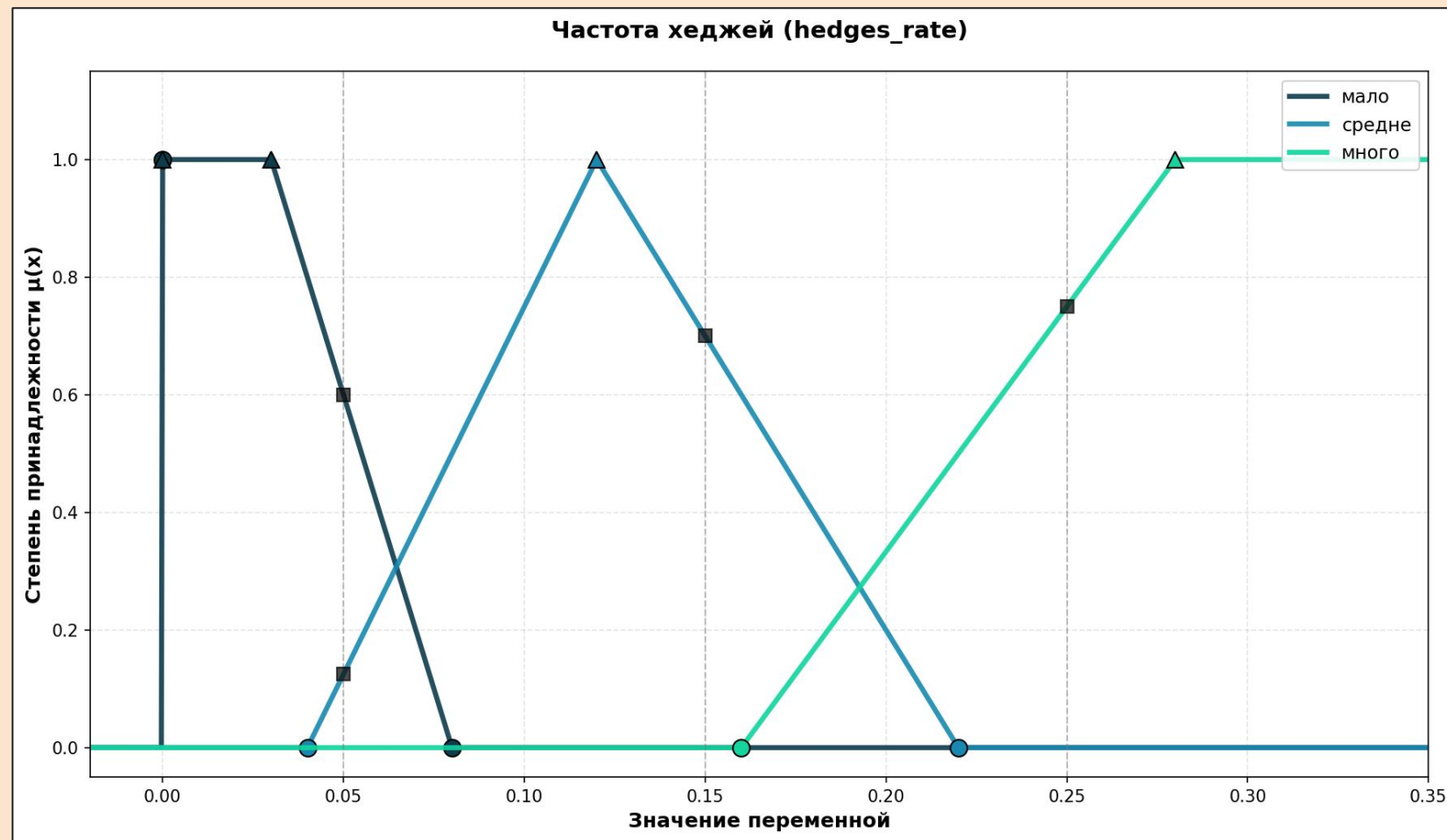
Нечёткая логика (fuzzy-ядро системы)

3. hedges_rate (Частота хеджей)

Диапазон: [0.0, 1.0]

Термы:

- мало: [0.0, 0.0, 0.03, 0.08]
- средне: [0.04, 0.12, 0.22]
- много: [0.16, 0.28, 1.0, 1.0]



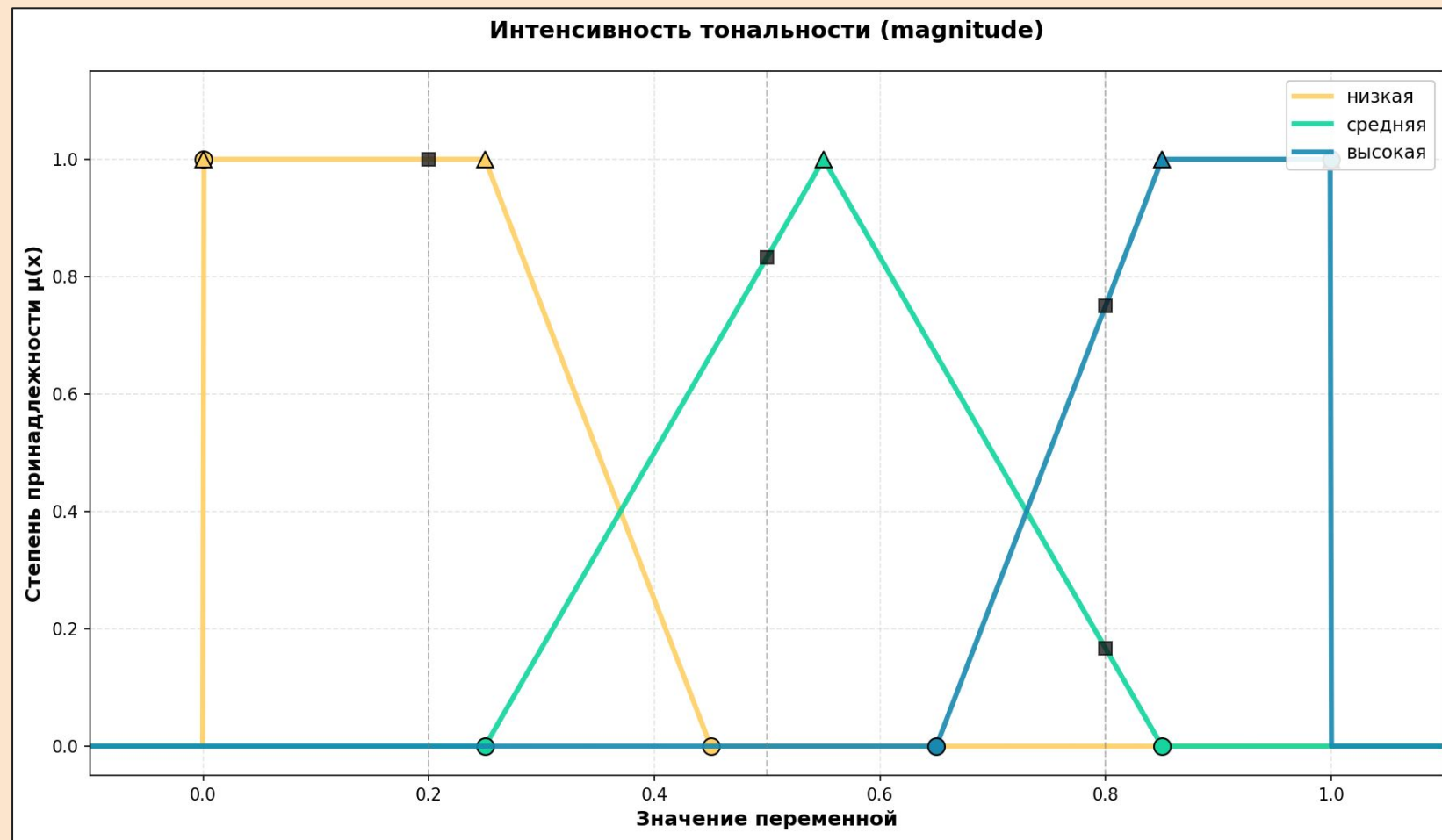
Нечёткая логика (fuzzy-ядро системы)

4. magnitude (интенсивность тональности)

Диапазон: [0.0, 1.0]

Термы:

- низкая: [0.0, 0.0, 0.25, 0.45]
- средняя: [0.25, 0.55, 0.85]
- высокая: [0.65, 0.85, 1.0, 1.0]



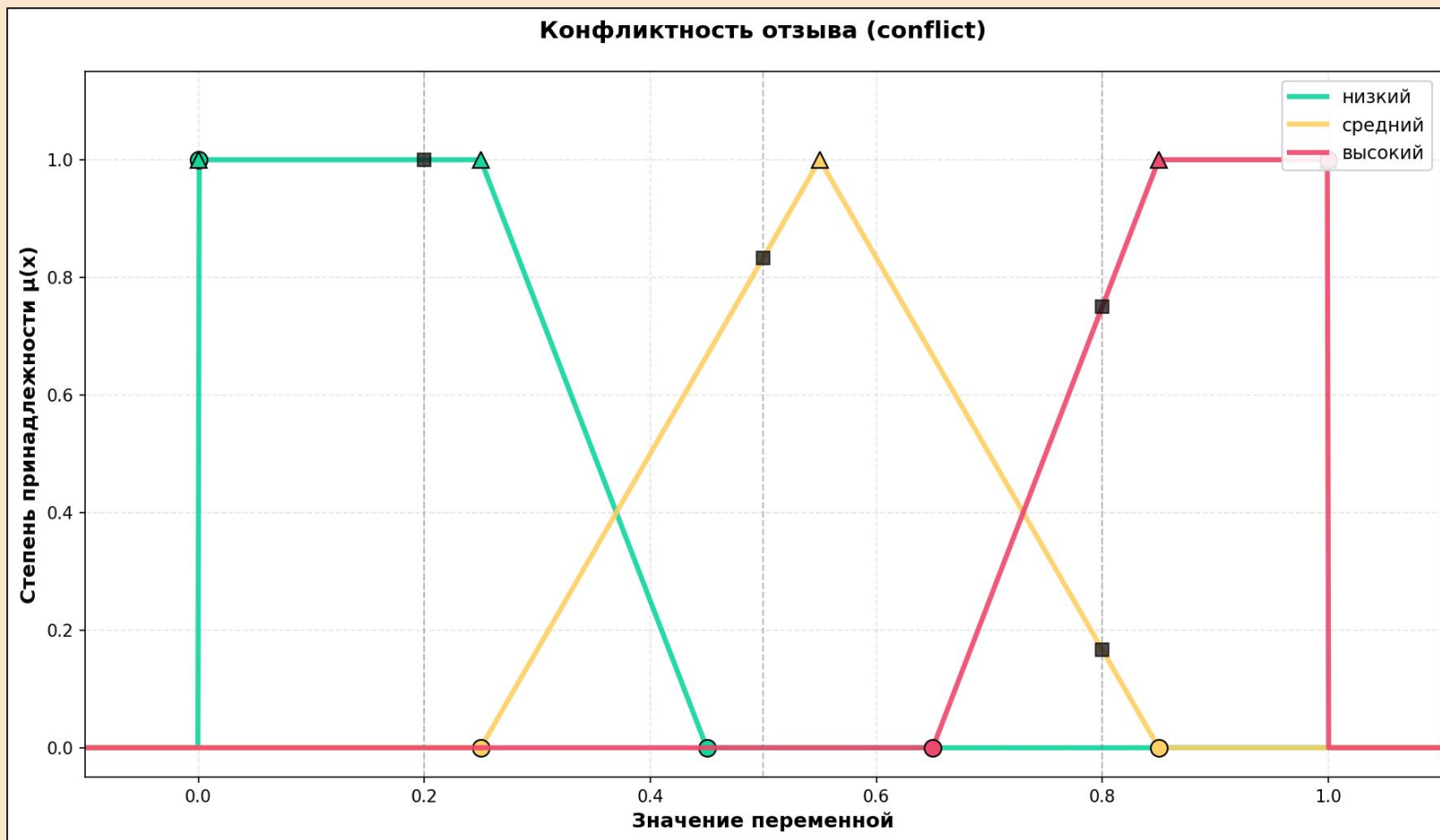
Нечёткая логика (fuzzy-ядро системы)

5. conflict (Конфликтность отзыва)

Диапазон: [0.0, 1.0]

Термы:

- низкий: [0.0, 0.0, 0.25, 0.45]
- средний: [0.25, 0.55, 0.85]
- высокий: [0.65, 0.85, 1.0, 1.0]



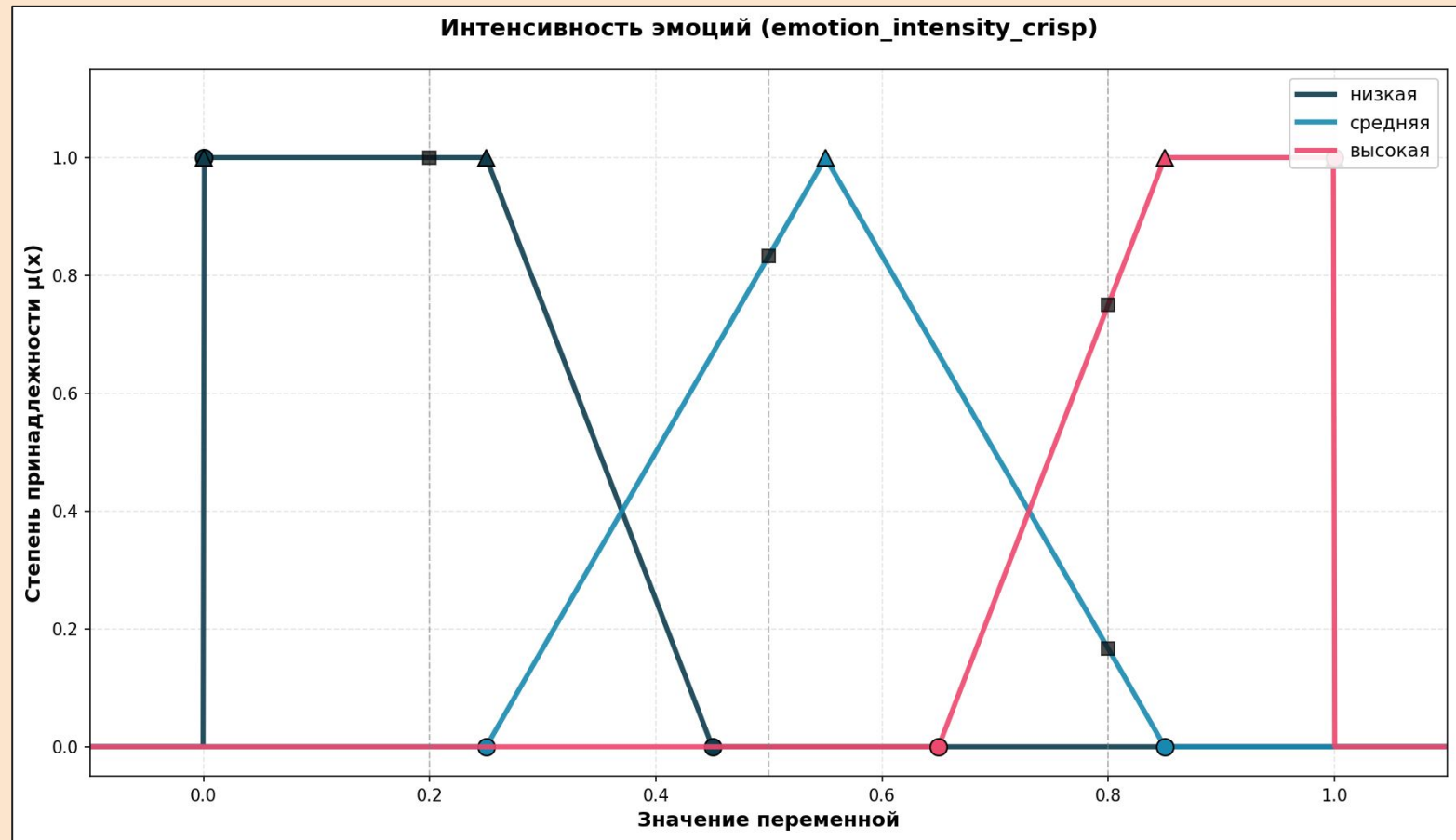
Нечёткая логика (fuzzy-ядро системы)

6. emotion_intensity_crisp (Интенсивность эмоций)

Диапазон: [0.0, 1.0]

Термы:

- низкая: [0.0, 0.0, 0.25, 0.45]
- средняя: [0.25, 0.55, 0.85]
- высокая: [0.65, 0.85, 1.0, 1.0]



Нечёткая логика (fuzzy-ядро системы)

Что делает:

- **Определяет математические структуры функций принадлежности:**
 - Triangle (треугольная функция принадлежности)
 - Trapezoid (трапециевидная функция принадлежности)
- **Реализует функции принадлежности**
- **Вычисляет производные признаки:**
 - hedges_rate - частота хэджеров
 - magnitude - интенсивность тональности
 - conflict - уровень конфликта (противоречивости отзыва)
- **Определяет входные лингвистические переменные (фаззификаторы):**
 - score_crisp - общая тональность [-1, 1]
 - coverage - покрытие лексиконом
 - hedges_rate - частота ослабителей
 - magnitude - магнитуа тональности
 - conflict - уровень конфликта
 - emotion_intensity_crisp - интенсивность эмоций

Нечёткая логика (fuzzy engine)

1. Фаззификация входных данных:

- Преобразует четкие входные признаки в нечеткие значения принадлежности

2. Нечеткий вывод:

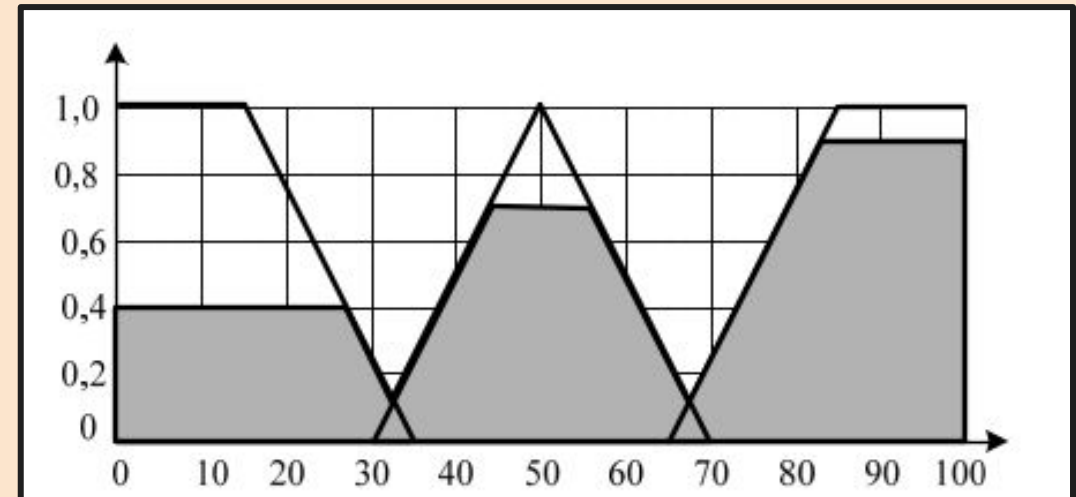
- Применяет набор правил
- Вычисляет силу активации каждого правила
- Агрегирует выходные нечеткие множества

3. Дефаззификация:

- Преобразует нечеткие выходные множества в четкие значения
- Использует метод центра тяжести

4. Формирование результата:

- Возвращает структурированный результат
- Включает промежуточные данные для отладки и анализа



Примеры (позитивный)

```
python3 -m fuzzy.run_stage3 --features
feature_create/my_features.jsonl --id 0 --pretty --top-k-rules 5
--show-text
```

FUZZY REVIEWS — Stage 3 report (single review)
id: 0

TEXT

"кофе отличный, обслуживание очень хорошее и уютная атмосфера."

CRISP FEATURES (stage 2 + derived)

score_crisp	0.578298
emotion_intensity_crisp	0.502095
coverage	0.75
hedges_count	0
tokens_alpha	8
pos	2.52145
neg	0.6737
magnitude	1.000
conflict	0.422
hedges_rate	0.000

other: {"id": 0, "text": "кофе отличный, обслуживание очень хорошее и уютная атмосфера.", "intensity": 0.532525, "emotion_intensity_score": 0.502095, "lex_hits": 6, "intensifiers_count": 1, "downtoners_count": 0, "negations_count": 0, "contrast_count": 0}

FUZZY INPUTS (μ)

score_crisp:
позитивная 0.91 | сильно_позитивная 0.09
coverage:
высокое 0.50 | среднее 0.33
hedges_rate:
мало 1.00
magnitude:
высокая 1.00
conflict:
средний 0.57 | низкий 0.14
emotion_intensity_crisp:
высокая 1.00

OUTPUT AGGREGATION (μ)

trust:
высокое 0.50 | среднее 0.33
final_tonality:
позитивная 0.91 | сильно_позитивная 0.09
final_emotion_intensity:
высокая 1.00
impression_strength:
высокая 0.57

DEFUZZIFIED OUTPUTS (crisp)

final_tonality_crisp 0.534
trust 0.684
final_emotion_intensity 0.869
impression_strength 0.852

TOP RULES (contribution)

[1.00] R23_emo_base_high: final_emotion_intensity=высокая
[1.00] R25_emo_boost_emo_high_mag_high:
final_emotion_intensity=высокая
[0.91] R13_ton_base_score_pos: final_tonality=позитивная
[0.57] R33_impr_high_mag_high_conf_med: impression_strength=высокая
[0.50] R07_trust_high_cov_high_hedges_low: trust=высокое

Примеры (негативный)

```
python3 -m fuzzy.run_stage3 --id 0 --pretty
```

FUZZY REVIEWS — Stage 3 report (single review)

id: 0

CRISP FEATURES (stage 2 + derived)

```
score_crisp      -0.183624
emotion_intensity_crisp 0.251713
coverage         0.708333
hedges_count     0
tokens_alpha     24
pos              1.84943
neg              2.6814
magnitude        0.539
conflict         0.816
hedges_rate      0.000
other: {"id": 0, "text": "качество плохое пошив ужасный
(горловина наперекос) Фото не соответствует Ткань ужасная
рисунок блеклый маленький рукав не такой УЖАС!!!! не стоит за
такие деньги г.....", "intensity": 0.266519,
"emotion_intensity_score": 0.251713, "lex_hits": 17,
"intensifiers_count": 0, "downtoners_count": 0, "negations_count": 3,
"contrast_count": 0}
```

FUZZY INPUTS (μ)

```
score_crisp:
  нейтральная 0.27 | негативная 0.08
coverage:
  среднее 0.47 | высокое 0.29
hedges_rate:
  мало 1.00
magnitude:
  средняя 0.96
conflict:
  высокий 0.83 | средний 0.11
emotion_intensity_crisp:
  низкая 0.99
```

OUTPUT AGGREGATION (μ)

```
trust:
  среднее 0.47 | высокое 0.29
final_tonality:
  негативная 0.08
final_emotion_intensity:
  низкая 0.99
impression_strength:
  средняя 0.96
```

DEFUZZIFIED OUTPUTS (crisp)

```
final_tonality_crisp -0.502
trust                0.623
final_emotion_intensity 0.183
impression_strength  0.550
```

TOP RULES (contribution)

```
[0.99] R17_int_low__emo_low_or_mag_low: final_emotion_intensity=низкая
[0.96] R19_impr_med__mag_med: impression_strength=средняя
[0.47] R03_trust_med__cov_med__hedges_low: trust=среднее
[0.29] R01_trust_high__cov_high__hedges_low: trust=высокое
[0.08] R11_ton_neg__score_neg__cov_high__hedges_low:
final_tonality=негативная
```

Примеры (должен быть негативный)

```
python3 -m fuzzy.run_stage3 --features
feature_create/my_features.jsonl --id 1 --pretty --top-k-rules 5
--show-text
```

FUZZY REVIEWS — Stage 3 report (single review)
id: 1

TEXT

"пошла аллергия, чуть не померла"

CRISP FEATURES (stage 2 + derived)

score_crisp	0.380196
emotion_intensity_crisp	0.142733
coverage	0.4
hedges_count	1
tokens_alpha	5
pos	0.2955
neg	0.1327
magnitude	0.245
conflict	0.620
hedges_rate	0.200

other: {"id": 1, "text": "пошла аллергия, чуть не померла",
"intensity": 0.2141, "emotion_intensity_score": 0.142733, "lex_hits":
2, "intensifiers_count": 0, "downtoners_count": 1, "negations_count":
1, "contrast_count": 0}

FUZZY INPUTS (μ)

score_crisp:
 позитивная 0.58
coverage:
 среднее 0.50 | низкое 0.25
hedges_rate:
 много 0.33 | средне 0.20
magnitude:
 низкая 1.00
conflict:
 средний 0.77
emotion_intensity_crisp:
 низкая 1.00

OUTPUT AGGREGATION (μ)

trust:
 низкое 0.35 | среднее 0.20
final_tonality:
 позитивная 0.58 | нейтральная 0.33
final_emotion_intensity:
 низкая 1.00
impression_strength:
 низкая 0.77

DEFUZZIFIED OUTPUTS (crisp)

final_tonality_crisp	0.348
trust	0.353
final_emotion_intensity	0.179
impression_strength	0.189

TOP RULES (contribution)

[1.00] R21_emo_base_low: final_emotion_intensity=низкая
[1.00] R24_emo_down__mag_low: final_emotion_intensity=низкая
[0.77] R27_impr_low__mag_low__conf_med: impression_strength=низкая
[0.58] R13_ton_base__score_pos: final_tonality=позитивная
[0.35] R06_trust_low__cov_med__hedges_high: trust=низкое

Примеры (должен быть позитивный)

```
python3 -m fuzzy.run_stage3 --features
feature_create/my_features.jsonl --id 0 --pretty --top-k-rules 5
--show-text
```

FUZZY REVIEWS — Stage 3 report (single review)
id: 0

TEXT

"очень вкусно и интересно! дети угощают всех. нашей бабушке 94 года, говорит - это просто шедевр!"

CRISP FEATURES (stage 2 + derived)

score_crisp	-0.86187
emotion_intensity_crisp	0.199088
coverage	0.571429
hedges_count	0
tokens_alpha	14
pos	0.1125
neg	1.5164
magnitude	0.332
conflict	0.138
hedges_rate	0.000

other: {"id": 0, "text": "очень вкусно и интересно! дети угощают всех. нашей бабушке 94 года, говорит - это просто шедевр!", "intensity": 0.203613, "emotion_intensity_score": 0.199088, "lex_hits": 8, "intensifiers_count": 1, "downtoners_count": 0, "negations_count": 0, "contrast_count": 0}

FUZZY INPUTS (μ)

score_crisp:	
сильно_негативная	1.00
coverage:	
среднее	0.93
hedges_rate:	
мало	1.00
magnitude:	
низкая	0.59 средняя 0.27
conflict:	
низкий	1.00
emotion_intensity_crisp:	
низкая	0.63 средняя 0.17

OUTPUT AGGREGATION (μ)

trust:	
среднее	0.93
final_tonality:	
сильно_негативная	1.00
final_emotion_intensity:	
низкая	0.63 средняя 0.17
impression_strength:	
низкая	0.59 средняя 0.27

DEFUZZIFIED OUTPUTS (crisp)

final_tonality_crisp	-0.838
trust	0.550
final_emotion_intensity	0.286
impression_strength	0.329

TOP RULES (contribution)

[1.00] R10_ton_base__score_strong_neg: final_tonality=сильно_негативная
[0.93] R04_trust_med__cov_med__hedges_low: trust=среднее
[0.63] R21_emo_base_low: final_emotion_intensity=низкая
[0.62] R24_emo_down__mag_low: final_emotion_intensity=низкая
[0.59] R26_impr_low__mag_low__conf_low: impression_strength=низкая

бай-бай кчау

